

Sky Analytics Stream

Architettura Distribuita per l'Analisi in Tempo Reale del Traffico Aereo

Flavio Simonelli

Matricola 0365333

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

flavio.simonelli@alumni.uniroma2.eu

Francesco Masci

Matricola 0365258

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

francesco.masci.02@alumni.uniroma2.eu

Sommario—Questo lavoro presenta *Sky Analytics Stream*, un'architettura distribuita per l'analisi e il monitoraggio in tempo reale del traffico aereo statunitense basata su Apache Flink e Apache Kafka. Il sistema riceve i dati da un simulatore ad alte prestazioni sviluppato in Go, in grado di riprodurre flussi di eventi storici in event-time con disordine controllato e fattori di accelerazione variabili. Attraverso un'unica topologia a grafo diretto aciclico, la pipeline esegue in parallelo tre analisi: aggregazioni orarie delle performance delle compagnie aeree, classifiche in tempo reale degli aeroporti con ritardi severi e stima dei percentili dei ritardi tramite l'algoritmo DDSketch. Risultati e metriche di telemetria sono storicizzati in InfluxDB e visualizzati tramite Grafana. La tolleranza ai guasti è garantita da checkpointing *exactly-once* con stato incrementale gestito da RocksDB. Il sistema, testato sia localmente che su cluster Amazon EC2, sostiene throughput superiori a 9.000 eventi/secondo. La valutazione sperimentale analizza le prestazioni del sistema al variare del parallelismo, del disordine degli eventi e della frequenza dei checkpoint.

Index Terms—Apache Flink, Stream Processing, Apache Kafka, Traffico Aereo, Percentili Approssimati, Tolleranza ai Guasti, DDSketch, checkpointing.

I. INTRODUZIONE

L'analisi tempestiva del traffico aereo richiede di elaborare flussi continui di dati per aggiornare metriche prestazionali e classifiche di ritardo in tempo reale. In questo contesto, l'accuratezza e la freschezza dei risultati dipendono dalla capacità di gestire le sfide intrinseche dello stream processing. In particolare, ritardi di rete, colli di bottiglia nell'ingestione e arrivi fuori ordine fanno sì che il tempo logico in cui un volo si verifica, ovvero l'*event-time*, non venga rispettato nell'ordine di come gli eventi arrivano ai processori di elaborazione. Per fornire risultati coerenti e completi, è dunque cruciale definire politiche di watermarking e di gestione dei dati tardivi che bilancino correttamente la latenza e l'accuratezza delle aggregazioni.

Per affrontare tali sfide, questo lavoro presenta il progetto e la valutazione sperimentale di *Sky Analytics Stream*, un'architettura distribuita progettata per elaborare un dataset storico di voli negli Stati Uniti (relativi al periodo gennaio–aprile 2025). Il dataset viene riprodotto in modo controllato tramite un simulatore ad alte prestazioni sviluppato in Go, in grado di iniettare dati in Apache Kafka applicando fattori di compressione temporale e introducendo disordine artificiale. La pipeline di elaborazione principale è realizzata in Apache

Flink e implementa un singolo grafo aciclico ottimizzato per eseguire in parallelo tre analisi distinte:

- 1) Il calcolo orario delle statistiche di volo per ciascuna compagnia aerea;
- 2) Il tracciamento dinamico delle classifiche Top-10 degli aeroporti con i maggiori ritardi accumulati;
- 3) La stima continua della distribuzione dei ritardi complessivi.

Il resto della relazione è organizzato come segue: la Sezione II descrive l'architettura generale del sistema e il flusso dei dati; la Sezione III approfondisce i dettagli implementativi delle singole componenti; la Sezione IV illustra l'infrastruttura di deployment; la Sezione V presenta la valutazione sperimentale e i risultati dei test prestazionali; infine, la Sezione VI conclude il report delineando possibili sviluppi futuri.

II. ARCHITETTURA DEL SISTEMA

L'architettura di *Sky Analytics Stream* è concepita come una pipeline multi-tier per lo stream processing distribuito, orientata ad alta scalabilità, robustezza e tolleranza ai guasti. Il sistema adotta un approccio a microservizi disaccoppiati in cui ciascun componente presiede a una specifica fase della pipeline: generazione, brokeraggio, elaborazione, persistenza e visualizzazione, come illustrato nello schema logico di Fig. 1.

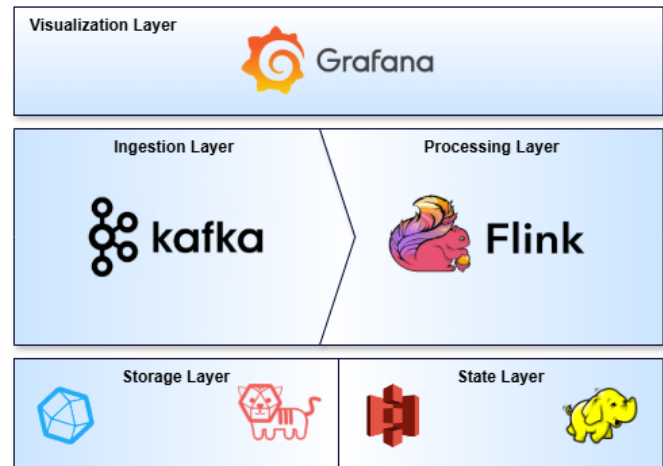


Figura 1. Rappresentazione dello stack tecnologico coinvolto in Sky Analytics Stream.

A. Stack Tecnologico

La pipeline si articola in sei livelli architetturali principali:

- 1) **Ingestion Layer:** Un modulo sviluppato in Go che simula la sorgente in tempo reale partendo dal dataset storico locale. Ottimizzato per alte prestazioni, funge da produttore Kafka.
- 2) **Message Broker Layer:** Il bus dei messaggi distribuito che funge da buffer. Disaccoppia la sorgente di dati dal motore di elaborazione e quest'ultimo dal consumatore Telegraf, attenuando i picchi di carico e gestendo in modo trasparente la *backpressure*.
- 3) **Stream Processing Layer:** Il nucleo computazionale del sistema. Esegue una topologia distribuita consumando i dati da Kafka ed eseguendo elaborazioni stateful in event-time.
- 4) **State Storage Layer:** Lo storage distribuito deputato al salvataggio dei checkpoint. RocksDB funge da backend di stato locale (in-memory/su disco locale del worker), mentre HDFS garantisce la persistenza distribuita dei backup di stato per la tolleranza ai guasti.
- 5) **Storage Layer:** InfluxDB memorizza sia i risultati analitici che le metriche di runtime di Flink. Telegraf funge da connettore reattivo leggendo i risultati da Kafka e inserendoli in InfluxDB.
- 6) **Visualization Layer:** Fornisce l'interfaccia utente finale per l'osservabilità delle performance del cluster e il monitoraggio degli indicatori analitici del traffico aereo.

B. Flusso dei Dati

Il ciclo di vita di un singolo evento di volo attraversa sequenzialmente l'intera pipeline (mostrata in Fig. 1) seguendo il flusso descritto di seguito:

- 1) **Generazione e Replay:** Il simulatore Go legge i dati grezzi in streaming direttamente dal dataset compresso. Gli eventi vengono ordinati per data/ora logica di volo, convertiti in JSON e inviati al topic Kafka `flights-stream` con una frequenza che rispetta lo *Speedup Factor* impostato. Qualora configurato, viene applicato l'algoritmo di sfasamento temporale per inserire disordine nello stream.
- 2) **Elaborazione:** I TaskManager di Apache Flink consumano lo stream di eventi. Il motore gestisce i watermark in base all'event-time inserito nei record e calcola i risultati per le tre query analitiche concorrenti. Man mano che le finestre temporali si chiudono, Flink emette i risultati intermedi e finali, scrivendoli in specifici topic Kafka di output (es. `flights-q1-results`, `flights-q2-1h-results`, ecc.).
- 3) **Fault-Tolerance:** Durante l'elaborazione, Flink esegue periodicamente checkpoint asincroni. Lo stato locale delle finestre e degli sketch viene sincronizzato su HDFS/S3.
- 4) **Raccolta e Persistenza:** Telegraf ascolta i topic di output di Kafka e converte i messaggi JSON in record pronti per InfluxDB, inserendoli nel bucket dei risultati.

Parallelamente, il reporter InfluxDB integrato in Flink invia ogni secondo le metriche di telemetria ad un bucket separato in InfluxDB.

- 5) **Visualizzazione:** Grafana interroga costantemente InfluxDB tramite il linguaggio Flux/InfluxQL, aggiornando in tempo reale le visualizzazioni grafiche delle dashboard destinate sia alle metriche applicative (es. ritardi delle compagnie) sia a quelle infrastrutturali (es. latenza end-to-end e carico del cluster).

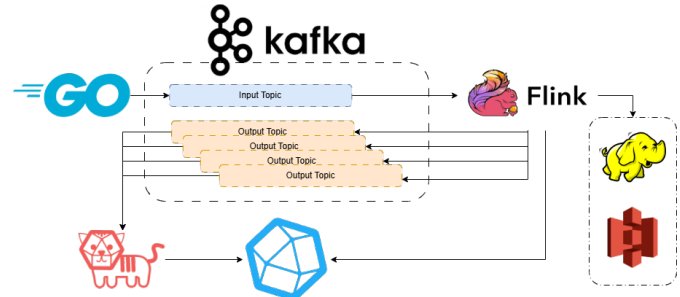


Figura 2. Diagramma di interazione tra le componenti principali della pipeline di Sky Analytics Stream.

III. IMPLEMENTAZIONE DELLE COMPONENTI

A. Simulatore di Ingestione in Go

L'ingestione e il replay degli eventi storici in tempo reale sono gestiti da un componente custom sviluppato in linguaggio Go. La scelta tecnologica è motivata dalla necessità di disporre di un agente di ingestione ad altissime prestazioni, caratterizzato da un consumo minimo di risorse e in grado di garantire throughput elevati senza introdurre colli di bottiglia o latenze spurie dovute alla Garbage Collection (frequenti in ecosistemi basati su JVM).

Per simulare scenari di esecuzione realistici e testare i limiti prestazionali del sistema, sono stati definiti tre scenari di carico basati su diversi fattori di accelerazione temporale (*Speedup Factor*), descritti nella Tabella I.

Si evidenzia che l'uso dell'accelerazione temporale per aumentare il throughput rappresenta una forzatura metodologica: comprimere il tempo logico riduce drasticamente la durata reale delle finestre temporali di Flink (fino a soli 83 millisecondi nello Scenario C), imponendo una frequenza di trigger e di calcolo dello stato estremamente elevata e artificiale. In uno scenario industriale, qualora si volesse studiare il comportamento del sistema sotto carichi massivi senza tale distorsione, la soluzione ottimale consisterebbe nel sintetizzare dati aggiuntivi (generando nuovi record fittizi all'interno dello stesso intervallo temporale) in modo da incrementare il throughput mantenendo la durata reale delle finestre vicina a quella reale.

In base a tali scenari di accelerazione, è possibile stimare teoricamente il throughput massimo in uscita atteso per ciascuna delle tre query analitiche (OUT_i), assumendo che non si verifichino scarti o ritardi tardivi tali da innescare

Tabella I
SCENARI DI CARICO E ACCELERAZIONE DEL SIMULATORE GO

Parametro / Metrica	Sec. A (Basso)	Sec. B (Medio)	Sec. C (Alto)
Moltiplicatore target	1.440x	14.400x	43.200x
Fattore tempo log/real	1g / 1min	10g / 1min	30g / 1min
Durata del Replay nominale	≈ 120 min	≈ 12 min	≈ 4 min
Throughput nominale	≈ 305 ev/s	≈ 3.050 ev/s	≈ 9.150 ev/s
Finestra 1h reale	2,5 s	0,25 s	0,083 s
Efficienza reale	≈ 99%	≈ 96%	≈ 89% – 91%
Throughput reale misurato	≈ 302 ev/s	≈ 2.928 ev/s	≈ 8.140 – 8.320 ev/s

emissioni supplementari (Tabella II). La stima si basa sul numero massimo di chiavi attive e sulla frequenza di chiusura o trigger delle finestre in tempo reale:

- **Query 1:** Emette 4 record (uno per ciascuna compagnia target) alla chiusura di ogni finestra logica di 1 ora.
- **Query 2:** Consiste di tre finestre concorrenti. La finestra da 1 ora e la finestra globale (avente trigger periodico a 1 ora logica) emettono al massimo 10 record ciascuna; la finestra da 6 ore emette 10 record ogni 6 ore logiche (media di 1.67 record/ora). Il totale teorico è dunque di 21.67 record per ora logica.
- **Query 3:** Composta da tre finestre. Le finestre da 1 giorno e globale (con trigger a 1 giorno logico) emettono al massimo 96 record ciascuna (4 compagnie × 24 ore); la finestra da 7 giorni emette 96 record ogni 7 giorni logici (media di 13.71 record/giorno). Il totale è di 205.71 record per giorno logico.

Tabella II
THROUGHPUT MASSIMO TEORICO E REALE IN USCITA ATTESO PER CIASCUNA QUERY

Query / Pipeline	Sec. A (1.440x)	Sec. B (14.400x)	Sec. C (43.200x)
Query 1	1,60 recs/s	16,00 recs/s	48,00 recs/s
Query 2 (1h + 6h + Global)	8,67 recs/s	86,67 recs/s	260,00 recs/s
Query 3 (1d + 7d + Global)	3,43 recs/s	34,29 recs/s	102,86 recs/s
Totale Max Nominale (OUT)	13,70 recs/s	136,96 recs/s	410,86 recs/s
Totale Reale Atteso (OUT)	13,56 recs/s	131,48 recs/s	365,66 – 373,88 recs/s

Si osserva che nella realtà sperimentale, il throughput di Query 2 può risultare inferiore a causa della condizione sul traffico minimo (≥ 30 voli in event-time) che esclude molti aeroporti a bassa frequenza dall'ordinamento Top-10. Al contrario, il throughput reale di Query 3 (o Query 1) può talvolta registrare fluttuazioni positive rispetto a quello teorico a causa dei record tardivi (*late records*) che, se rientranti nei margini di *allowed lateness*, innescano aggiornamenti e pubblicazioni incrementali delle finestre Flink precedentemente chiuse.

Il simulatore implementa diverse funzionalità avanzate e ottimizzazioni ingegneristiche:

- **Ingestione Zero-Copy:** Al primo avvio, il simulatore tenta di localizzare l'archivio del dataset (in formato `.tar.gz`) in una directory locale configurata tramite la variabile d'ambiente `INPUT_ARCHIVE_PATH`. Qualora non presente, l'applicazione lo scarica automaticamente

tramite richiesta HTTP dal server del dipartimento¹. Successivamente, viene verificata l'integrità del file confrontandone l'hash SHA-1 con quello atteso. Una volta superato il controllo, l'archivio originale viene scompattato al volo in memoria tramite i pacchetti nativi di Go, senza la necessità di estrarre preventivamente su disco l'intero dataset in formato CSV. Questo flusso zero-copy riduce al minimo l'uso dello spazio di archiviazione ed elimina i tempi morti dovuti alle operazioni di I/O su disco.

- **Binary Caching:** Al fine di minimizzare i tempi di inizializzazione nelle successive esecuzioni, gli eventi letti dal file CSV vengono convertiti in una slice di strutture tipizzate Go di tipo `models.FlightRecord`. I record vengono quindi ordinati cronologicamente rispetto al loro event-time logico e serializzati in un file di cache binario locale `.gob` utilizzando il codificatore nativo `encoding/gob` di Go.
- **Gestione del Tempo Fisico:** Per ciascun volo, l'event-time logico viene ricostruito combinando i campi `YEAR`, `MONTH`, `DAY_OF_MONTH` e l'orario programmato `CRS_DEP_TIME`. Il simulatore emula la distanza temporale reale tra i voli applicando un fattore di compressione del tempo ed attendendo la differenza relativa riscalata rispetto all'evento precedente. Con alti fattori di accelerazione (dove le finestre temporali logiche di un'ora si contraggono in pochi millisecondi reali), le funzioni di *sleep* del sistema operativo risulterebbero inadeguate a causa dell'overhead dello scheduler, del kernel e dei *context switch*. Per ovviare a ciò, è stata implementata una strategia di attesa ibrida (*Sleep-Spinlock*): se l'attesa supera la soglia critica *spin threshold* nel file di configurazione, il thread esegue una *sleep* cooperativa rilasciando la CPU, mentre al di sotto di tale valore effettua un **spinlock**. Questo impedisce la sospensione del processo da parte del kernel nel momento critico, garantendo precisioni nell'ordine dei microsecondi ed assicurando l'uniformità del throughput programmato.
- **Out-of-Order Injection:** Al fine di stressare e validare le strategie di Watermarking e la gestione dei dati tardivi di Apache Flink e analizzare il sistema in scenari realistici dove le sorgenti sono diversificate e poco affidabili, il simulatore implementa un decoratore basato su un algoritmo probabilistico. Con una probabilità definita $P \in [0, 1]$, un record viene selezionato e il suo istante di pubblicazione viene dislocato in avanti nel tempo di un offset casuale limitato da un valore massimo configurabile in minuti logici. La sequenza di eventi risultante viene infine ri-ordinata sul tempo di pubblicazione reale prima dell'invio, riproducendo accuratamente i tipici disallineamenti di rete di uno stream distribuito.
- **Pubblicazione su Kafka:** I record sono serializzati in JSON mantenendo i nomi delle colonne e inviati al topic `flights-stream` mediante `kafka-go`. Il writer usa

¹<http://www.ce.uniroma2.it/courses/sabd2526/project/project-1-data.tar.gz>

bilanciamento round-robin, batch fino a 100 messaggi o 100 kB, timeout di 2 ms e modalità asincrona, evitando che la conferma di ogni singolo messaggio alteri la temporizzazione del replay. In alternativa, un sink testuale consente test locali senza Kafka per sviluppo e debugging.

B. Event Bus

Per disaccoppiare la sorgente dei dati (il simulatore Go) dal motore di stream processing (Apache Flink), è stato integrato **Apache Kafka** come message broker distribuito ad alto throughput. Kafka funge da polmone architetturale, assorbendo le fluttuazioni di carico del flusso in ingresso ed esponendo i dati in modo ordinato e persistente.

1) Organizzazione dei Topic e Partizionamento:

Il flusso di dati in ingresso è gestito tramite il topic principale `flights-stream`, configurato con 6 partizioni parallele. Il partizionamento consente di parallelizzare l'elaborazione su più slot di esecuzione di Flink, garantendo la scalabilità orizzontale del sistema. Per la storizzazione dei risultati analitici prodotti dalle tre query concorrenti, sono stati definiti diversi topic di output dedicati per ciascuna finestra e granularità temporale (es. `flights-q1-results`, `flights-q2-1h-results`, `flights-q3-1d-results`, ecc.). Questo disaccoppia ulteriormente Flink dai database di storage a valle, permettendo a componenti ausiliari come Telegraf di consumare i risultati in modalità asincrona.

2) *Fase di Ingestione in Flink (Kafka Source)*: L'integrazione di Kafka come sorgente dati in Flink avviene tramite la classe nativa `KafkaSource`, istanziata mediante un builder personalizzato (`SourceBuilder`). Il consumo dello stream parte dall'offset meno recente (`OffsetsInitializer.earliest()`).

Per mappare i byte grezzi transitati su Kafka in oggetti Java tipizzati, è stato implementato uno schema di deserializzazione custom, `FlightRecordDeserializationSchema` (che implementa l'interfaccia `KafkaRecordDeserializationSchema<FlightRecord>`). Tale classe esegue le seguenti operazioni per ogni record consumato:

- **Deserializzazione JSON**: Utilizza una libreria Jackson `ObjectMapper` (dichiarata `transient` e istanziata `lazy` per evitare errori di serializzazione nei `TaskManager`) per decodificare il payload del messaggio in un oggetto `FlightRecord`, scartando in modo tollerante eventuali record corrotti o messaggi vuoti (tombstone);
- **Estrazione dell'Event-Time**: Calcola l'epoch millisecondi logico associato al volo (`eventTimeMillis`) combinando i campi `data` e `ora` del record;
- **Iniezione del Tempo di Ingestione**: Inietta nel record il timestamp fisico corrente del sistema (`systemIngestionTime`) tramite `System.currentTimeMillis()`. Questo passaggio è cruciale per misurare accuratamente la latenza end-to-end della pipeline a valle.

3) *Fase di Scrittura dei Risultati (Kafka Sink)*: Le tre query analitiche concorrenti producono flussi di risultati che vengono re-iniettati in Kafka tramite istanze di `KafkaSink`, configurate con la garanzia di consegna `DeliveryGuarantee.AT_LEAST_ONCE` per garantire la tolleranza ai guasti senza l'overhead delle transazioni a due fasi.

La serializzazione da oggetti Java a payload JSON pronti per essere trasmessi in rete è delegata a schemi di serializzazione personalizzati (come `AirlinePerformanceRecordSerializer` per la Query 1 e `DelayDistributionRecordSerializer` per la Query 3). Ognuno di essi serializza lo stato dell'oggetto in array di byte JSON. Nel caso della Query 1, il codice della compagnia aerea (`OP_UNIQUE_CARRIER`) viene utilizzato esplicitamente come chiave di partizionamento del record Kafka. Questo assicura che gli eventi relativi allo stesso vettore vengano inoltrati alla medesima partizione di Kafka, agevolando eventuali letture downstream ordinate o partizionate per chiave.

C. Motore di Stream Processing: Apache Flink

Il core computazionale di *Sky Analytics Stream* è realizzato in **Apache Flink** ed è strutturato in un singolo job distribuito. La topologia a grafo diretto aciclico (DAG) è orchestrata all'interno della classe principale `FlightAnalysisJob` e prevede la condivisione a monte della sorgente dei dati e dei filtri di data quality. Questo design a sorgente condivisa riduce al minimo l'I/O ed evita la deserializzazione ridondante dello stream di ingresso.

1) *Gestione del Disordine Temporale (Out-of-Ordering e Watermarking)*: La riproduzione accelerata dei dati introduce disallineamenti cronologici logici che richiedono strategie di sincronizzazione robuste per l'avanzamento del tempo logico (Event-Time):

- **Generazione dei Watermark**: Viene adottata una `WatermarkStrategy` configurata con la politica di disordine massimo limitato (`forBoundedOutOfOrderness`), parametrizzata tramite il file `application.yaml` (es. 15 o 30 minuti logici).
- **Idleness Detection**: Per evitare che il blocco o il rallentamento di una delle 6 partizioni di Kafka arresti l'avanzamento del watermark globale dell'intera pipeline, la strategia è irrobustita dalla clausola `withIdleness(Duration.ofMinutes(1))`, che esclude temporaneamente dal calcolo le partizioni inattive.
- **Dati Tardivi (Allowed Lateness) e Side Output**: A livello di finestra (Query 1 e 2), viene applicato il metodo `allowedLateness(Duration)` per consentire l'aggiornamento tardivo delle metriche qualora giungessero eventi con un ritardo superiore al watermark ma inferiore alla tolleranza aggiuntiva. Per gli eventi che superano anche questa seconda barriera, Flink utilizza un canale di **Side Output** dedicati (`sideOutputLateData`) per

reindirizzare tali record a un analizzatore personalizzato (LateRecordMetricAnalyzer), impedendo che vengano scartati silenziosamente o che provochino la corruzione dei dati delle finestre chiuse.

2) *Meccanismo di Checkpointing e Fault Tolerance*: La tolleranza ai guasti e la resilienza del cluster EC2 sono gestite tramite il meccanismo di checkpointing distribuito asincrono di Flink (basato sull'algoritmo di Chandy-Lamport):

- **Consistenza Exactly-Once**: Il motore è impostato per garantire che lo stato interno degli operatori venga ripristinato esattamente nello stesso stato pre-fallimento (*exactly-once*), allineando le barriere di checkpoint lungo i canali di rete del DAG.
- **State Backend incrementale con RocksDB**: Per evitare colli di bottiglia causati dalla memoria heap della JVM, lo stato in-flight delle finestre è ospitato su disco locale tramite il backend **RocksDB**. Quest'ultimo scrive le tabelle di stato in file SST (*Sorted String Table*) locali ed effettua checkpoint **incrementali asincroni**. Ad ogni intervallo di checkpoint, solo i byte modificati (*delta*) vengono caricati nello storage durevole centralizzato (HDFS locale o Amazon S3 in cloud), garantendo tempi di salvataggio rapidi anche per stati di grandi dimensioni.
- **Ottimizzazioni anti-saturazione**: La proprietà `MIN_PAUSE_BETWEEN_CHECKPOINTS` assicura un intervallo di riposo minimo tra la fine di un checkpoint e l'inizio del successivo, eliminando il rischio di saturazione della CPU (checkpoint backpressure).

3) *Garanzie di Consegna E2E: Exactly-Once Interno vs At-Least-Once su Kafka*: Una delle decisioni architetturali più rilevanti del progetto riguarda il disallineamento pianificato delle garanzie di consegna:

- **La scelta di At-Least-Once su Kafka**: Il `SinkBuilder` configura i producer Kafka impostando `DeliveryGuarantee.AT_LEAST_ONCE`. L'implementazione di un sink *exactly-once* richiederebbe l'attivazione del protocollo a transazioni distribuite di Kafka (Two-Phase Commit), il quale introduce un forte overhead di coordinamento e costringe i consumatori a valle (Telegraf) a leggere i messaggi in modalità `read_committed`, bloccandone l'ingestione fino al completamento del checkpoint di Flink successivo e innalzando la latenza end-to-end.
- **Risoluzione End-to-End Exactly-Once tramite Idempotenza**: Questa perdita teorica di consistenza sui sink viene completamente compensata a valle da **InfluxDB**. Poiché InfluxDB è un database temporale che memorizza i punti indicizzati per timestamp e tag-set primari, eventuali messaggi duplicati scritti da Kafka a causa di un ripristino di Flink verranno sovrascritti in modo identico (*write idempotency*). Questo schema ibrido (Exactly-Once distribuito all'interno di Flink + At-Least-Once su Kafka + Scritture Idempotenti su InfluxDB) permette di ottenere la consistenza matematica dell'Exactly-

Once end-to-end senza pagare il costo prestazionale delle transazioni distribuite.

4) *L'importanza del componente DDSketch per i Percentili e la Latenza*: L'algoritmo probabilistico **DDSketch** è una componente strategica dell'intera architettura, impiegato per risolvere problemi di stima quantilica a memoria costante $O(1)$:

- **Caratteristiche dell'Algoritmo ed Evidenze Scientifiche**: DDSketch mappa i valori su indici di bin logaritmici basati su una precisione relativa garantita $\alpha = 0.01$ (errore massimo dell'1%). Come evidenziato nel paper originale di Datadog [2]:

"...in practice m [the number of bins] is usually set to be a constant large enough to handle a wide range of values. As an example, for $\alpha = 0.01$, a sketch of size 2048 can handle values from 80 microseconds to 1 year, and cover all quantiles."

In conformità con questa analisi teorica, tutte le implementazioni degli accumulatori DDSketch nel nostro progetto (sia analitici che di telemetria) sono state configurate impostando la dimensione massima dello store di bin (`MAX_STORE_BINS`) a $m = 2048$ e l'accuratezza relativa a $\alpha = 0.01$. Questo ci consente di coprire l'intera gamma dei possibili ritardi dei voli e tempi di calcolo degli operatori (da frazioni di millisecondo fino a anomalie di scala annuale) a memoria strettamente limitata e senza che la struttura debba mai ricorrere al collasso prematuro dei bin.

- **Dualità d'uso nel Sistema**: DDSketch è stato impiegato con successo in due contesti chiave:

- 1) **Query 3 (Distribuzione dei Ritardi)**: All'interno dell'accumulatore `DelayDistributionAccumulator`, DDSketch stima in tempo reale e senza salvare i singoli record in memoria la distribuzione dei ritardi delle compagnie per ciascuna fascia oraria su scale giornaliere, settimanali e globali.
- 2) **Metriche di Latenza Custom (ProcessingLatencyTracker)**: Per misurare in tempo reale l'efficienza degli operatori e la latenza end-to-end (E2E) della pipeline, è stato implementato un tracciante di performance che invia le misurazioni di latenza (in frazioni di millisecondo ottenute via `System.nanoTime()`) ad accumulatori basati su DDSketch. Questo ci consente di pubblicare su Grafana metriche altamente accurate sui percentili della latenza (p50, p75, p90, p99) con un impatto nullo sulla memoria dei nodi `TaskManager`.

5) *Sistema di Telemetria e Metriche Custom (Direct-to-InfluxDB)*: L'osservabilità e il monitoraggio delle performance del sistema in tempo reale si basano su un set di metriche personalizzate registrate tramite la Flink Metric API. Tali metriche si dividono in tre categorie principali:

- **Metriche di Data Quality:** Il contatore `corrupted_records_total` incrementa ad ogni violazione delle regole di integrità e null-safety nel filtro di preprocessing (`LogicalInconsistencyFilter`).
- **Metriche di Record Tardivi (Lateness):** All'interno dell'operatore di side-output `LateRecordMetricAnalyzer`, vengono registrati il contatore `total_late_records_count` (quantità di voli arrivati oltre la tolleranza temporale), il gauge di picco storico `max_lateness_minutes_absolute` (il ritardo logico massimo assoluto riscontrato su un singolo evento tardivo) e i percentili stimati di latenza dell'event-time (`lateness_p50`, `p90`, `p95`, `p99`) calcolati tramite uno sketch `DDSketch`.
- **Metriche di Processing e Latenza E2E:** Misurate in ogni operatore chiave tramite la classe `ProcessingLatencyTracker`. Esse includono sia la latenza interna dell'operatore (espressa in millisecondi in virgola mobile tramite `System.nanoTime()` per eliminare la granularità grossolana dei millisecondi e l'impatto del clock skew distribuito), sia la latenza end-to-end (E2E) dei dati, stimata su tre orizzonti diversi: rispetto al record più vecchio nella finestra (`e2e_oldest`), rispetto al record più recente (`e2e_newest`), e rispetto all'istante medio di Ingestion della finestra (`e2e_avg`).

Architettura di Flusso Direct-to-InfluxDB per la Telemetria: A differenza dei risultati di business delle query (che transitano su Kafka per garantire persistenza, tolleranza ai guasti e disaccoppiamento dei database), le metriche di telemetria hanno un ciclo di vita transitorio ed elevata frequenza di aggiornamento. Di conseguenza, non necessitano di un canale persistente basato su Kafka, il quale comporterebbe solo un inutile overhead di rete ed un inquinamento dei topic con dati estranei all'analitica.

Per avviare a ciò, il `JobManager` e i `TaskManager` di Flink sono configurati nei file di deploy per caricare la classe nativa `InfluxdbReporterFactory`. Flink stabilisce connessioni HTTP dirette verso la porta 8086 di InfluxDB, inviando periodicamente in modo asincrono le telemetrie di sistema e le metriche custom. Questa soluzione bypassa interamente Kafka per i dati di monitoraggio, preservando la larghezza di banda del broker per il flusso di ingestion primario e riducendo al minimo la latenza di aggiornamento sulle dashboard di Grafana.

6) *Purezza del Modello Dati e Ottimizzazione della Serializzazione POJO:* Per ottimizzare lo scambio di dati sulla rete e le performance di scrittura sul backend di stato (RocksDB), è stata condotta una ristrutturazione architetturale del modello dati principale:

- **Disaccoppiamento del Modello:** La classe `FlightRecord` è stata spogliata da qualsiasi dipendenza dalle librerie esterne di Kafka e Apache Flink, isolando lo schema di deserializzazione in un file autonomo (`FlightRecordDeserializationSchema`).

Questo ha reso `FlightRecord` un **POJO (Plain Old Java Object)** puro secondo le convenzioni di Java.

- **Vantaggi prestazionali di Flink:** Flink utilizza un analizzatore di tipi a runtime. Se una classe rispetta le regole formali dei POJO (costruttore vuoto pubblico, attributi non-final e metodi getter/setter per ciascun campo), Flink evita di utilizzare la serializzazione generica Kryo, la quale memorizza l'intero grafo di classe producendo payload voluminosi e costosi in termini di CPU.
- **Generazione di PojoSerializer:** Flink genera per i POJO dei serializzatori nativi speciali (`PojoSerializer`), che serializzano solo i byte grezzi dei dati in modo posizionale fisso. Questo produce due grandi benefici:
 - 1) **Riduzione delle dimensioni dello stato:** Lo stato archiviato in RocksDB è notevolmente più compatto, riducendo i tempi di I/O per i checkpoint asincroni.
 - 2) **Ottimizzazione dei Shuffle di Rete:** Durante le fasi di `keyBy`, lo shuffle dei record tra i `TaskManager` su EC2 avviene al massimo throughput consentito dalla rete, riducendo l'utilizzo di CPU per la codifica/decodifica.

7) *Dettagli delle Query Analitiche ed Ingegnierizzazione del DAG:*

- **Query 1 (Performance Compagnie Aeree):** Filtra i record sui quattro vettori primari tramite `TargetAirlineFilter` ed effettua una proiezione a basso footprint per ridurre la dimensione dei record prima dello shuffle. Utilizza un aggregatore incrementale (`AirlinePerformanceAggregator`) per aggiornare lo stato in tempo costante $O(1)$ all'interno di una finestra tumbling di 1 ora in Event-Time.
- **Query 2 (Classifica Aeroporti per Ritardi Severi):** Calcola contemporaneamente la top-10 su finestre di 1 ora, 6 ore e globale. Adotta una strategia di classificazione ottimizzata in due fasi logiche: un'aggregazione locale per aeroporto in finestra (`RankAirportsAggregator`), seguita da una partizione logica sul timestamp di fine della finestra (`windowEnd`) verso la classe `RankAirportsRankProcessor`. Quest'ultima raccoglie le metriche parziali in un `MapState` locale, registra un timer sul watermark e, al suo scattare, inserisce gli aeroporti idonei (traffico ≥ 30 voli) in un Min-Heap (coda con priorità di dimensione fissa 10) con tie-breaker deterministici (ritardi severi, ritardo medio, ID aeroporto). Lo stato locale viene ripulito immediatamente dopo l'emissione del risultato per evitare memory leak.
- **Query 3 (Distribuzione dei Ritardi):** Raggruppa gli eventi su chiave composita `Tuple2<Airline, DepartureHour>` per fasce orarie da 0 a 23 in finestre di 1 giorno, 7 giorni e globale, calcolando i percentili tramite il summenzionato modulo `DDSketch`.

IV. INFRASTRUTTURA DI DEPLOYMENT

La pipeline di *Sky Analytics Stream* è stata progettata per supportare due modalità di esecuzione speculari ma di-

mentionate diversamente: un deployment locale a container multiplo per sviluppo, e un deployment distribuito multi-nodo su infrastruttura cloud per test prestazionali ed esperimenti di scalabilità.

A. Deployment Locale

La configurazione locale consente di validare la correttezza logica del codice e testare il comportamento del sistema in scenari controllati. Viene realizzata tramite un file `docker-compose.yml` che orchestra undici container connessi a una rete privata bridge:

- **Ingestion & Broker:** Un container per il broker Kafka e un container di inizializzazione che crea i topic necessari impostandovi il rispettivo partizionamento (6 partizioni per lo stream di ingresso);
- **Stream Processing:** Un container Flink JobManager e tre container TaskManager. Ogni TaskManager dispone di due slot di elaborazione, per un parallelismo massimo locale di 6;
- **State Storage:** Tre container HDFS (un NameNode e due DataNodes) per supportare il salvataggio dei checkpoint in modalità distribuita locale;
- **Persistence & Telemetry:** Un container InfluxDB (versione 2.7) per storicizzare risultati e telemetrie, accoppiato a un container Telegraf che converte i messaggi JSON consumati da Kafka nel modello dati di InfluxDB;
- **Visualization:** Un container Grafana accoppiato a un container `grafana-image-renderer` (usato per esportare e catturare i grafici delle dashboard).

I parametri di configurazione (porte fisiche, credenziali di accesso ai database, token di telemetria e parametri di accelerazione) sono iniettati tramite variabili d'ambiente caricate da un file locale `.env`.

B. Deployment Distribuito su AWS EC2

Per condurre le campagne sperimentali in condizioni di carico reale, il sistema è distribuito su un cluster multi-nodo in ambiente cloud Amazon Web Services.

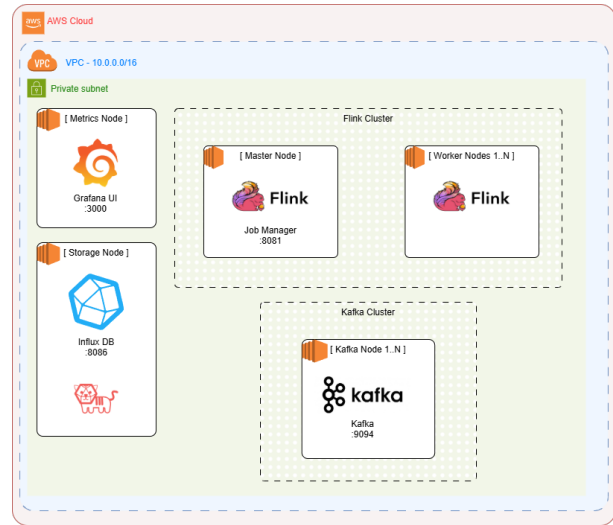


Figura 3. Vista architetturale del deployment distribuito su AWS EC2.

1) *Provisioning dell'Infrastruttura (CloudFormation & Route 53):* Il deployment su AWS è automatizzato tramite script che richiamano template di **AWS CloudFormation**. L'orchestratore esegue le seguenti fasi:

- 1) **Network Stack:** Crea una VPC dedicata, una sottorete pubblica con CIDR associato, una Security Group con regole di ingresso/uscita per le porte dei servizi, e una **Private Hosted Zone** su **Amazon Route 53** con dominio privato `flight-analysis.local`;
- 2) **Bootstrap ed S3:** I pacchetti pre-compilati, i file di configurazione e le variabili d'ambiente vengono caricati su un bucket **Amazon S3** privato;
- 3) **Node Provisioning:** Lancia le istanze EC2. Ciascun nodo esegue uno script di bootstrap che installa Docker, scarica le configurazioni e i pacchetti dal bucket S3 e avvia i container Docker locali. I nodi vengono registrati automaticamente su Route 53 per consentire la risoluzione dei nomi privati.

2) *Dimensionamento delle Istanze EC2:* Il cluster distribuito è composto da istanze EC2 appartenenti alla famiglia general-purpose `t3` e compute-optimized `c6i` di AWS², dimensionate per bilanciare i costi con la necessità di risorse CPU e memoria:

- **Kafka Cluster:**

- *Broker 1:* Istanza **t3.large** (2 vCPU, 8 GiB RAM) per gestire il carico di scrittura primario e avviare il reply del dataset;
- *Broker 2 & 3:* Istanze **2 x t3.medium** (2 vCPU, 4 GiB RAM) che agiscono come repliche e broker ausiliari.

- **Flink JobManager:** Istanza **c6i.large** (2 vCPU, 4 GiB RAM) ottimizzata per il calcolo;
- **Flink TaskManagers:** Istanze **3 x c6i.large** (2 vCPU, 4 GiB RAM) adibite all'esecuzione fisica dei task di Flink (ogni TaskManager ospita 2 slot di calcolo);

²<https://aws.amazon.com/it/ec2/pricing/on-demand/>

- **InfluxDB e Telegraf:** Istanza `t3.small` (2 vCPU, 2 GiB RAM) per il salvataggio persistente di metriche e risultati;
- **Grafana:** Istanza `t3.small` (2 vCPU, 2 GiB RAM) per ospitare il servizio di dashboarding;

La risoluzione dei nomi DNS privati consente alle macchine di dialogare in modo trasparente senza dover hardcodare gli indirizzi IP pubblici o privati variabili delle istanze.

V. VALUTAZIONE SPERIMENTALE

In questa sezione viene presentata la valutazione sperimentale dell'architettura *Sky Analytics Stream*. L'analisi delle prestazioni è suddivisa in tre campagne di test mirate a valutare: l'impatto del parallelismo e la scalabilità, la gestione del disordine temporale (out-of-orderness), e l'impatto del checkpointing con il relativo ripristino in caso di guasti. Tutti i test sono stati condotti sul cluster distribuito AWS EC2 descritto nella Sezione IV-B.

Nota sull'efficienza reale del simulatore: Si evidenzia che la velocità di replay effettiva del simulatore Go risente del carico computazionale, dei context switch e dei tempi di I/O del sistema operativo, in particolare sotto elevati fattori di accelerazione temporale. Dalle rilevazioni sperimentali, il simulatore Go raggiunge circa il **99%** dello speedup target nello scenario lento ($1.440\times$), circa il **96%** nello scenario medio ($14.400\times$) e oscilla tra l'**89%** e il **91%** nello scenario veloce ($43.200\times$). Di conseguenza, i throughput in ingresso effettivi misurati su Flink risultano proporzionalmente ridimensionati rispetto ai valori nominali teorici.

A. Primo Confronto: Parallelismo e Scalabilità in Assenza di Disordine

La prima campagna di test valuta il comportamento della pipeline al variare del livello di parallelismo ($P \in \{1, 3, 6\}$) in condizioni di dati ordinati all'origine (disordine aggiuntivo pari a 0 e tolleranza al ritardo impostata a 0). I test analizzano i due scenari a carico più sostenuto riprodotti dal simulatore Go: lo scenario a **Medio Carico** (moltiplicatore $14.400\times$, throughput nominale ≈ 3.050 eventi/secondo) e lo scenario ad **Alto Carico** o stress test (moltiplicatore $43.200\times$, throughput nominale ≈ 9.150 eventi/secondo). Le metriche prestazionali a livello di cluster e di nodo sono state raccolte da InfluxDB e visualizzate su Grafana.

1) *Analisi del Throughput e delle Latenze a Livello di Cluster:* L'analisi del throughput a livello di cluster evidenzia differenze significative tra i livelli di parallelismo, in particolare nello scenario a medio carico ($14.400\times$):

- **Parallelismo 1** ($P = 1$): Come si osserva in Fig. 4 (sinistra), il throughput in ingresso reale si attesta intorno a ≈ 2.400 eventi/secondo. Questo valore è inferiore al throughput nominale di 3.050 eventi/secondo generato dal simulatore, indicando che un singolo slot di calcolo non è in grado di sostenere il carico complessivo della pipeline, provocando un accumulo di ritardo (*lag*) nei topic di Kafka a monte.
- **Parallelismo 3 e 6** ($P \in \{3, 6\}$): Con l'aumento dei thread di elaborazione (Fig. 4, destra), il throughput in ingresso sale e si stabilizza perfettamente alla soglia nominale di ≈ 3.000 eventi/secondo. Questo dimostra che il cluster ha superato il collo di bottiglia e riesce a consumare i dati alla velocità di generazione del simulatore. Il throughput in uscita totale si attesta a circa 100 record/secondo complessivi (di cui ≈ 50 record/s per la Query 1, ≈ 30 record/s per la Query 2 e $\approx 15 - 18$ record/s per la Query 3).

Nello scenario ad alto carico ($43.200\times$), il throughput reale in ingresso si stabilizza per tutte le configurazioni di parallelismo a circa ≈ 8.100 eventi/secondo, leggermente al di sotto del valore nominale di 9.150 eventi/secondo. Questa limitazione non è imputabile a colli di bottiglia computazionali di Apache Flink, bensì a vincoli di I/O di rete o alla capacità di throughput del broker Kafka a singolo nodo nel nostro ambiente AWS EC2. Tuttavia, la latenza end-to-end media (*Latency Mark*) rimane estremamente stabile e controllata al di sotto dei 120 ms per la Query 2 Global (la più complessa), confermando la stabilità del motore di calcolo sotto carico pesante.

2) *Analisi del Consumo delle Risorse a Livello di Nodo:* L'incremento del parallelismo mostra un impatto chiaro e positivo sulla distribuzione del carico computazionale (CPU) tra le istanze EC2 attive:

- **Parallelismo 1** ($P = 1$): L'intera elaborazione del DAG di Flink viene allocata su un singolo slot di calcolo di un unico TaskManager. Nello scenario a medio carico, l'istanza `flink-worker-1` registra un uso di CPU del $\approx 12 - 16\%$, mentre gli altri worker rimangono inattivi ad uno 0% reale. Nello stress test ad alto carico ($43.200\times$, Fig. 5 a sinistra), l'unico worker attivo (`flink-worker-3`) subisce un picco di utilizzo di CPU che oscilla tra il 60% e il 70% , rappresentando un punto singolo di potenziale instabilità per l'infrastruttura.
- **Parallelismo 3** ($P = 3$): Flink alloca i 3 slot attivi distribuendo il carico su due TaskManager: `flink-worker-3` registra circa il 45% di CPU nello stress test, mentre `flink-worker-2` lavora al 22% CPU. Il terzo worker rimane inutilizzato.
- **Parallelismo 6** ($P = 6$): Con tutti i 6 slot del cluster attivi (2 slot per ciascuno dei 3 TaskManager), il carico viene bilanciato in modo eccellente tra tutte e tre le macchine. Nello scenario a medio carico la CPU si attesta uniformemente tra il $11 - 13\%$, mentre nello stress test (Fig. 5 a destra) tutte le istanze lavorano in modo cooperativo e simmetrico con un consumo di CPU stabile intorno al 25% .

Questo comportamento evidenzia la scalabilità orizzontale dell'applicazione: all'aumentare del parallelismo, Flink distribuisce i sotto-task e le relative finestre di calcolo in modo bilanciato su tutti i nodi del cluster, riducendo drasticamente il carico sui singoli TaskManager ed eliminando i picchi di utilizzo della CPU.



Figura 4. Throughput del cluster per lo scenario a medio carico ($14.400\times$) con parallelismo $P = 1$ (sinistra) e $P = 3$ (destra).

VI. CONCLUSIONI E SVILUPPI FUTURI

RIFERIMENTI BIBLIOGRAFICI

- [1] L. Fernando, H. Bindra, and K. Daudjee, “An experimental analysis of quantile sketches over data streams,” in *Proceedings of the 26th International Conference on Extending Database Technology (EDBT '23)*, pp. 1–12, 2023.
- [2] C. Masson, J. E. Rim, and H. K. Lee, “DDSketch: A fast and fully-mergeable quantile sketch with relative-error guarantees,” *Proceedings of the VLDB Endowment*, vol. 12, no. 12, pp. 2195–2205, 2019.

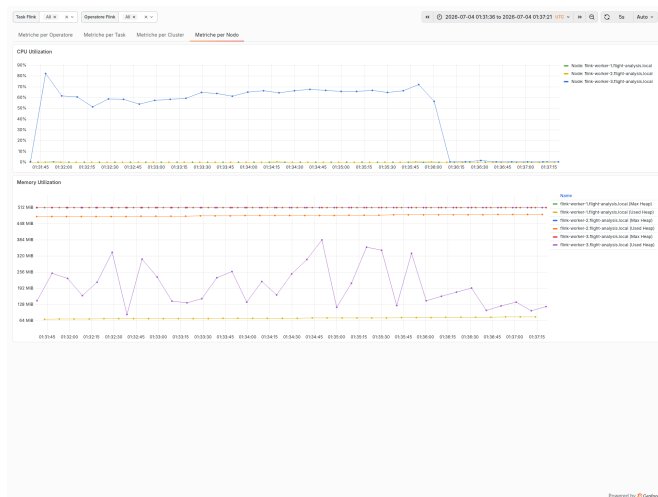


Figura 5. Utilizzo della CPU sui nodi TaskManager nello stress test ($43.200\times$) con parallelismo $P=1$ (sinistra) e $P=6$ (destra).

APPENDICE A
UTILIZZO DI STRUMENTI DI INTELLIGENZA ARTIFICIALE GENERATIVA

APPENDICE B
RAPPRESENTAZIONI GRAFICHE E ALLEGATI